CORE MATHEMATICS & ALGORITHMS

Sum of Divisors Formula

The Sum of Divisors Formula comes directly from prime factorization. Let's understand why it works instead of just memorizing it.

Example: $n = 12$

First, find the prime factorization:

$$12 = 2^2 \times 3^1$$

Now list all divisors of 12:

1 2 3 4 6 12

Sum:

$$1 + 2 + 3 + 4 + 6 + 12 = 28$$

Now let's see how the formula gives the same answer.

Step 1: Think about the powers of each prime

For the prime 2, the exponent is 2. So while making a divisor, you can choose:

- $2^0 = 1$
- $2^1 = 2$
- $2^2 = 4$

Possible choices:

1, 2, 4

For the prime 3, the exponent is 1. Choices are:

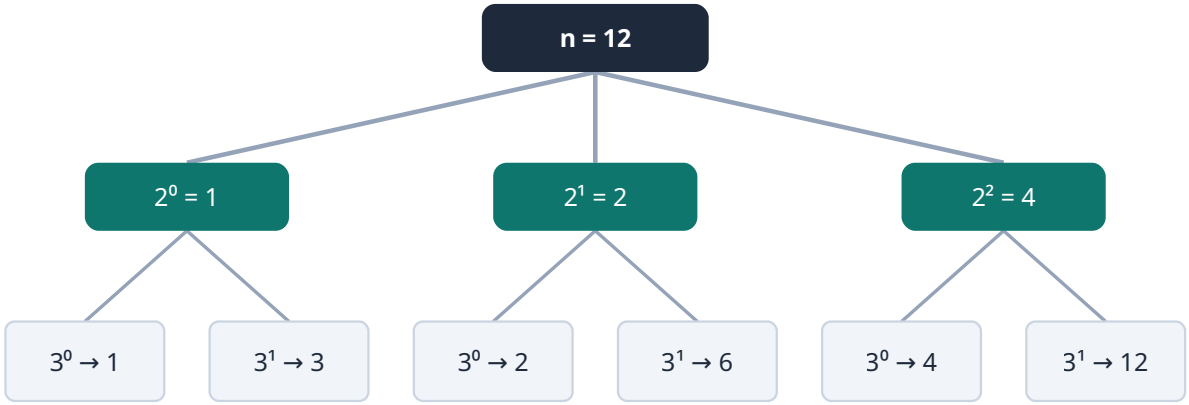
- $3^0 = 1$
- $3^1 = 3$

Possible choices:

1, 3

Step 2: Build every divisor

Every divisor is made by choosing one power of 2 and one power of 3.



Power of 2	Power of 3	Divisor
1	1	1
2	1	2
4	1	4
1	3	3
2	3	6
4	3	12

These are exactly the divisors:

1 **2** **3** **4** **6** **12**

Step 3: Instead of generating them, multiply the sums

Instead of writing every divisor, observe:

Choices for 2:

$$1 + 2 + 4$$

Choices for 3:

$$1 + 3$$

Multiply them:

$$(1 + 2 + 4) \times (1 + 3)$$

Expand:

$$= 1\times1 + 2\times1 + 4\times1 + 1\times3 + 2\times3 + 4\times3$$
$$= 1 + 2 + 4 + 3 + 6 + 12$$
$$= 28$$

Notice what happened?
Every product formed is exactly one divisor of 12.

Why does multiplication work?

Suppose

$$n = 2^2 \times 3^1 \times 5^1$$

A divisor is formed by choosing

- one power of 2
- one power of 3
- one power of 5

For example,

$$2^1 \times 3^0 \times 5^1 = 2 \times 1 \times 5 = 10$$

Another:

$$2^2 \times 3^1 \times 5^0 = 4 \times 3 \times 1 = 12$$

Every possible combination gives one divisor. So instead of generating every combination manually, we multiply the sums of possible powers.

General Formula

If

$$n = {p_1}^a \times {p_2}^b \times {p_3}^c$$

then the sum of all divisors is

$$(1 + p_1 + {p_1}^2 + \ldots + {p_1}^a)$$
$$\times$$
$$(1 + p_2 + {p_2}^2 + \ldots + {p_2}^b)$$

$$\times$$
$$(1 + p_3 + p_3^2 + \ldots + p_3^c)$$

Another Example

Take

$$n = 18$$

Prime factorization:

$$18 = 2^1 \times 3^2$$

Formula:

$$(1 + 2)$$
$$\times$$
$$(1 + 3 + 9)$$

Compute:

$$3 \times 13 = 39$$

Check manually:

Divisors:

1 2 3 6 9 18

Sum:

$$1 + 2 + 3 + 6 + 9 + 18 = 39$$

Correct!

Intuition to Remember

Think of each prime factor as giving you choices:

- For 2^2, choose: 1, 2, or 4.
- For 3^1, choose: 1 or 3.

Every divisor is created by picking one choice from each prime's list. When you multiply the sums of these choices, the distributive property automatically generates every possible combination exactly once. Since

each combination is one divisor, their total is the sum of all divisors. That's the key intuition behind the formula—it isn't magic; it's simply the distributive property generating every valid divisor.

Let's code it step by step using the prime factorization algorithm you've already learned.

Step 1: Formula

If

$$n = p_1^a \times p_2^b \times \dots$$

then

$$\textit{Sum of Divisors} = (1 + p_1 + p_1^2 + \dots + p_1^a) \times (1 + p_2 + p_2^2 + \dots + p_2^b) \times \dots$$

So during factorization, for every prime factor:

- Find its exponent.
- Compute the geometric sum.
- Multiply it into the answer.

Step 2: Factorize

Suppose

$$n = 360$$

Prime factorization

$$2^3 \times 3^2 \times 5^1$$

For 2^3
Need

$$1 + 2 + 4 + 8 = 15$$

For 3^2

Need

$$1 + 3 + 9 = 13$$

For 5^1

Need

$$1 + 5 = 6$$

Final answer

$$15 \times 13 \times 6 = 1170$$

Step 3: How do we compute $1+p+p^2+...$

There are two ways.

Method 1 (Easy)

Build it using multiplication.

```
long long sum = 1;
long long power = 1;
for(int i = 1; i <= cnt; i++)
{
    power *= prime;
    sum += power;
}
```

Example

$prime = 2$

$cnt = 3$

Initially

$sum = 1$

$power = 1$

Loop

$power = 2$

$sum = 3$

Loop

$power = 4$

$sum = 7$

Loop

power = 8

sum = 15

Done.

Step 4: Complete Code

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

ll sumOfDivisors(ll n)
{
    ll ans = 1;

    for(ll i = 2; i * i <= n; i++)
    {
        if(n % i == 0)
        {
            int cnt = 0;

            while(n % i == 0)
            {
                cnt++;
                n /= i;
            }

            ll sum = 1;
            ll power = 1;

            for(int j = 1; j <= cnt; j++)
            {
                power *= i;
                sum += power;
            }

            ans *= sum;
        }
    }

    if(n > 1)
    {
        ans *= (1 + n);
    }

    return ans;
}

int main()
{
    ll n;
    cin >> n;

    cout << sumOfDivisors(n);
}
```

Dry Run

Input

$$12$$

Initially

$$ans = 1$$

Prime

$$2$$

Count

$$2$$

Compute

$sum = 1, power = 1$
$power = 2, sum = 3$
$power = 4, sum = 7$

Now

$$ans = 7$$

Remaining

$$3$$

Since

$$n > 1$$

Multiply

$$ans *= 4$$

Result

$$28$$

Correct.

Time Complexity

Prime factorization

$$O(\sqrt{n})$$

Computing powers

The total number of exponent iterations across all prime factors is at most $O(\log n)$ because each division in the factorization removes one prime factor.

Overall:

$$O(\sqrt{n})$$

Space:

$$O(1)$$

A Cleaner Mathematical Version (Optional)

Instead of the inner loop, you can use the geometric series formula:

$$1 + p + p^2 + \cdots + p^k = \frac{p^{k+1} - 1}{p - 1}$$

However, in competitive programming, the iterative method shown above is often preferred because:

- it avoids floating-point issues from $pow()$,
- it's easy to understand,
- and it naturally fits into the prime factorization loop.